YEDİTEPE UNIVERSITY

T.C. YEDİTEPE UNIVERSITY

MTH424 - Generative AI Models

SPRING 2025

Term Project

Student Information	
Name and Surname	**No**
Mehmet Önal	20200702081
Oğuz Kağan Çayır	20210701002
Esra Kaya	20200702007
Ahmet Hasan Okumuş	20210701003

Introduction

In this project, the goal is to evaluate an open-source LLM's "prompt only" capabilities at text classification. The chosen task is "emotion classification". After selecting an English-language dataset from Huggingface, we applied classical machine learning methods and used Qwen to classify the emotions.

1. Dataset Description

The "dair-ai/emotion" dataset was chosen for the purpose of the project. This dataset has 20000 data and is already splitted as 16000 training data, 2000 validation data and 2000 test data. The dataset includes 6 classes or labels which are 0 for sadness, 1 for joy, 2 for love, 3 for anger, 4 for fear and 5 for surprise. This is not a balanced data set since the classes are included in different proportions. The detailed information of the dataset is as shown in *Table 1* and some examples of each training, validation and test set are shown in *Figure 1*, *Figure 2* and *Figure 3*, respectively.

Table 1: Emotion Classification Dataset

		Training		**Validation**		**Test**	
Class	**Label**	**# of Ex.**	**Percentage**	**# of Ex.**	**Percentage**	**# of Ex.**	**Percentage**
Sadness	0	4666	%29.2	550	%27.5	695	%34.8
Joy	1	5362	%33.5	704	%35.2	581	%29.1
Love	2	1304	%8.2	178	%8.9	159	%8
Anger	3	2159	%13.5	275	%13.8	275	%13.8
Fear	4	1937	%12.1	212	%10.6	224	%11.2
Surprise	5	572	%3.6	81	%4.1	66	%3.3
Total		16000	%100	2000	%100	2000	%100

```
df_train.head()
```

	text	label
0	i didnt feel humiliated	0
1	i can go from feeling so hopeless to so damned...	0
2	im grabbing a minute to post i feel greedy wrong	3
3	i am ever feeling nostalgic about the fireplac...	2
4	i am feeling grouchy	3

Figure 1: Training Set Examples

```
df_val.head()
```

	text	label
0	im feeling quite sad and sorry for myself but ...	0
1	i feel like i am still looking at a blank canv...	0
2	i feel like a faithful servant	2
3	i am just feeling cranky and blue	3
4	i can have for a treat or if i am feeling festive	1

Figure 2: Validation Set Examples

```
df_test.head()
```

	text	label
0	im feeling rather rotten so im not very ambiti...	0
1	im updating my blog because i feel shitty	0
2	i never make her separate from me because i do...	0
3	i left with my bouquet of red and yellow tulip...	1
4	i was feeling a little vain when i did this one	0

Figure 3: Test Set Examples

2. Classical Machine Learning Approach

2.1 TFIDF + Logistic Regression

Next step, the *scikit-learn* library used for *TFIDF* and *logistic regression* to train a model. The data was adapted using the *vectorizer* function of this library and the *logistic regression* model was applied using this data. After the model was trained, the model was used to predict the emotions on both validation and test set and *classification_report* function was used to evaluate the accuracy, precision, recall and F1-scores. The results are shown in *Figure 4* and *Figure 5*.

	precision	recall	f1-score	support
0	0.86	0.94	0.90	550
1	0.84	0.95	0.89	704
2	0.88	0.62	0.73	178
3	0.91	0.81	0.86	275
4	0.86	0.73	0.79	212
5	0.85	0.58	0.69	81
accuracy			0.86	2000
macro avg	0.87	0.77	0.81	2000
weighted avg	0.86	0.86	0.85	2000

Figure 4: Evaluation Metrics of the Validation Set

	precision	recall	f1-score	support
0	0.89	0.93	0.91	581
1	0.83	0.96	0.89	695
2	0.82	0.58	0.68	159
3	0.90	0.81	0.85	275
4	0.87	0.78	0.82	224
5	0.84	0.47	0.60	66
accuracy			0.86	2000
macro avg	0.86	0.75	0.79	2000
weighted avg	0.86	0.86	0.86	2000

Figure 5: Evaluation Metrics of the Test Set

2.2 TFIDF + Support Vector Classification (SVC)

Then, it was decided to use another model and *Support Vector Classification (SVC)* was applied to the both validation and test set. After the predictions, the same *classification_report* function of the library was used to evaluate the results and they were as shown in *Figure 6* and *Figure 7*, respectively.

	precision	recall	f1-score	support
0	0.86	0.93	0.89	550
1	0.82	0.96	0.89	704
2	0.93	0.58	0.71	178
3	0.92	0.78	0.85	275
4	0.85	0.75	0.80	212
5	0.93	0.64	0.76	81
accuracy			0.86	2000
macro avg	0.89	0.77	0.82	2000
weighted avg	0.86	0.86	0.85	2000

Figure 6: Evaluation Metrics of the Validation Set

	precision	recall	f1-score	support
0	0.89	0.92	0.90	581
1	0.81	0.96	0.88	695
2	0.88	0.53	0.66	159
3	0.91	0.79	0.84	275
4	0.87	0.79	0.83	224
5	0.86	0.45	0.59	66
accuracy			0.85	2000
macro avg	0.87	0.74	0.78	2000
weighted avg	0.86	0.85	0.85	2000

Figure 7: Evaluation Metrics of the Test Set

3. Using Qwen2.5-1.5B and a System Prompt

For the large language model (LLM) evaluation, **Qwen2.5-1.5B** model was used. The *AutoTokenizer* method from the Hugging Face *transformers* library was utilized. The tokenizer was initialized with the pre-trained weights of the **Qwen/Qwen2.5-1.5B** model.

After defining the *valid emotions* and creating the *emotion vector* corresponding to the validation set, *classify_batch_with_prompt* function was defined to classify the texts. This function was given a prompt as follows:

"You are a helpful assistant. Classify the following sentence into one of the following emotions: sadness, joy, love, anger, fear, surprise.

Sentence: "{text}"
Emotion: "

This function converts these prompts to inputs using the *tokenizer* which was initialized before. Then it generates the outputs using these inputs. After generating the outputs, it converts tokens to the text format. In the beginning, since the results could contain undefined labels such as “fully surprise” instead of “surprise”, the function was refined by cleaning the text output and including the just emotion token.

The function was used to classify emotions one text at a time in the beginning. The classification process was too slow, therefore it was decided to use a batch size of 32. This change has sped up the process. Then the predictions and valid emotions were compared using *classification_report* function. The results are shown in *Figure 8*.

	precision	recall	f1-score	support
sadness	0.45	0.86	0.59	92
joy	0.86	0.38	0.53	128
love	0.21	0.45	0.29	33
anger	0.40	0.26	0.32	38
fear	0.73	0.26	0.39	42
surprise	0.20	0.07	0.11	14
accuracy			0.48	347
macro avg	0.48	0.38	0.37	347
weighted avg	0.60	0.48	0.47	347

Figure 8: Evaluation Metrics of Qwen with zero-shot on the Validation Set

The confusion matrix was created based on the results and was analyzed to identify the most frequent misclassifications. The matrix is shown in *Figure 9*.

Hatalı Tahmin Karışıklık Sayıları:

predicted_label	anger	fear	joy	love	sadness	surprise	unknown
true_label							
anger	0	2	2	6	18	0	237
fear	2	0	0	7	22	0	170
joy	7	1	0	32	38	1	576
love	3	0	4	0	10	1	145
sadness	3	1	0	7	0	2	458
surprise	0	0	2	3	8	0	67

Figure 9: Confusion Matrix of Qwen with zero-shot on the Validation Set

An Alternative Approach: The Few-Shot Prompting Technique

Since the results are not satisfactory, the *few-shot* technique was used to enhance the classification performance. The prompt was updated as follows:

"You are a helpful assistant for classifying text into one of the following emotions:
sadness, joy, love, anger, fear, surprise.

You MUST respond with ONLY one of these words: sadness, joy, love, anger, fear, surprise.
No explanation. No other words.

Examples:
Sentence: "I feel so happy right now!"
Emotion: joy
Sentence: "This is the worst day of my life."
Emotion: sadness
Sentence: "I am so angry I could scream."
Emotion: anger
Sentence: "That sudden noise scared me half to death!"
Emotion: fear
Sentence: "I am deeply in love with this person."
Emotion: love
Sentence: "Wow, that was totally unexpected!"
Emotion: surprise
Sentence: "{text}"
Emotion: "

The results were evaluated again using *classification_report* function and the confusion matrix was generated to analyze the new predictions. The results are shown in *Figure 10* and *Figure 11*, respectively.

	precision	recall	f1-score	support
anger	0.68	0.39	0.49	275
fear	0.71	0.31	0.43	212
joy	0.83	0.30	0.44	704
love	0.24	0.36	0.29	178
sadness	0.45	0.87	0.59	550
surprise	0.18	0.33	0.24	81
unknown	0.00	0.00	0.00	0
accuracy			0.47	2000
macro avg	0.44	0.36	0.35	2000
weighted avg	0.61	0.47	0.47	2000

Figure 10: Evaluation Metrics of Qwen with few-shot on the Validation Set

Hatalı Tahmin Karışıklık Sayıları (Valid Predictions Only):

predicted_label	anger	joy	love	sadness	surprise	unknown
true_label						
anger	0	0	0	7	2	256
fear	0	0	1	3	1	196
joy	0	1	1	13	2	638
love	1	0	0	2	0	160
sadness	1	0	1	3	0	466
surprise	0	0	1	0	0	79

Figure 11: Confusion Matrix of Qwen with few-shot on the Validation Set

The model was applied to the validation set so far. Next, the model was used to predict the emotions on the test set. The accuracy, precision, recall and F1-score results and the confusion matrix were shown in *Figure 12* and *Figure 13*, respectively.

	precision	recall	f1-score	support
sadness	0.30	0.58	0.40	542
joy	0.41	0.15	0.22	699
love	0.06	0.11	0.08	174
anger	0.15	0.08	0.10	271
fear	0.08	0.03	0.05	205
surprise	0.04	0.07	0.05	81
accuracy			0.24	1972
macro avg	0.17	0.17	0.15	1972
weighted avg	0.26	0.24	0.22	1972

Figure 12: Evaluation Metrics of Qwen with few-shot on the Test Set

4. Extra Credit

4.1 BERT Embedding + Logistic Regression

In this section, an alternative way was tried for emotion classification. A classical machine learning approach using pre-trained BERT embeddings was tested. The embedding extraction was done with the *transformers* library. Then, a *logistic regression* model was trained with these embeddings using the *scikit-learn* library. After training, the model was evaluated on the validation and test set. The figures below show the accuracy, precision, recall and F1-score for each emotion class for both validation and test sets, respectively.

🔹 [Embedding + LogisticRegression] Validation Set Sonuçları:

	precision	recall	f1-score	support
sadness	0.58	0.68	0.63	550
joy	0.67	0.77	0.72	704
love	0.50	0.26	0.34	178
anger	0.53	0.39	0.45	275
fear	0.50	0.48	0.49	212
surprise	0.46	0.28	0.35	81
accuracy			0.60	2000
macro avg	0.54	0.48	0.50	2000
weighted avg	0.59	0.60	0.58	2000

Figure 13: Evaluation Metrics of the Validation Set

🔷 [Embedding + LogisticRegression] Test Set Sonuçları:

	precision	recall	f1-score	support
sadness	0.61	0.68	0.64	581
joy	0.69	0.79	0.73	695
love	0.29	0.18	0.22	159
anger	0.53	0.42	0.47	275
fear	0.51	0.42	0.46	224
surprise	0.40	0.33	0.36	66
accuracy			0.60	2000
macro avg	0.51	0.47	0.48	2000
weighted avg	0.58	0.60	0.59	2000

Figure 14: Evaluation Metrics of the Test Set

4.2 Fine Tuning a Small Language Model

In order to obtain better results, *Bert-Base-Uncased* model and *transformers* library were used and standard fine-tuning process was followed.

First, the training, validation, and test data was converted into the proper format. Then, the text data was tokenized using the BERT tokenizer. After that, the BERT model was fine-tuned using the *Trainer* API. The model was trained for 3 epochs, with a batch size of 16. After training, we used the model to predict emotions on the test set. The results were evaluated using accuracy, precision, recall and F1-score for each class. The results are shown in *Figure 15*.

	precision	recall	f1-score	support
sadness	0.96	0.98	0.97	581
joy	0.95	0.96	0.95	695
love	0.88	0.80	0.84	159
anger	0.94	0.91	0.93	275
fear	0.87	0.90	0.89	224
surprise	0.69	0.76	0.72	66
accuracy			0.93	2000
macro avg	0.88	0.88	0.88	2000
weighted avg	0.93	0.93	0.93	2000

Figure 15: Evaluation Metrics of the Fine-tuned BERT Model on the Test Set

5. Comparison and Analysis

In this project, we used a lot of techniques and trained different models. In this section, these approaches and the results will be discussed.

5.1 Classical Machine Learning Methods

Firstly, TFIDF method was used combined with logistic regression. This approach gave very good results in terms of the accuracy, precision, recall and F1-score on both the validation and the test set. The worst results were for the *surprise* class. It was not a surprise since the dataset contains at least *surprise* class data. When the results were evaluated, the accuracy, precision, recall and F1-score for label 5, which indicates the *surprise* class, are 0.86, 0.84, 0.47 and 0.60, respectively on the test set while they are 0.86, 0.85, 0.58 and 0.69, respectively on the validation set. Similar results can be observed for label 2 which is *love* class. This is the class with the fewest examples in the dataset after the *surprise*. Its recall value is 0.62 while the macro average is 0.77 and its F1-score is 0.73 while the macro average is 0.81. This shows that the lack of data can cause the model not to predict well even though the model can classify the other data satisfactorily.

Secondly, it was decided to use another model to observe the possible changes and maybe improvements. This time, TFIDF method was used combined with Support Vector Classification (SVC). After the same training and testing process, almost nothing has changed. The differences between the macro and weighted averages on the test set of these two models are at the 0.01 level. This demonstrates that both of these models work well but they don't give a solution to the data deficiency problem.
The model can be improved by adding more data especially for the *surprise* and *love* classes. In other words, the more balanced the dataset the better the results. But it should be noted that the more data can cause an overfitting problem and the data set should be adjusted by taking into account the evaluation metrics of other classes.

5.2 Using Qwen2.5-1.5B and a System Prompt

In this approach, a large language model (LLM), Qwen, was used for the purpose of emotion classification. This method allows us to utilize an advanced model which was trained for not only classification but also generating output.

Initially, the zero-shot approach was used to classify emotions using Qwen. The given prompt only included an explanation of the role of the LLM and how it would classify the texts without any example. The results are not as good as the ones of classical machine learning approaches.

All of the evaluation metrics have significantly decreased compared to the classical machine learning methods. Macro averages of the accuracy, precision, recall and F1-scores are 0.48, 0.48, 0.38 and 0.37, respectively. Some values are extremely low, especially for the *surprise* and the *love* classes while some evaluation metrics are decent such as recall of the *sadness* which is 0.86 or the precision of the *joy* which is also 0.86. When the confusion matrix is analyzed, it can be observed that only 1 validation example with the true label *surprise* was correctly predicted as *surprise*. Even though it is directly related to the number of examples of each class in the dataset, when the other results are considered, it indicates that the model has great difficulty or hesitations in choosing one of the 6 emotions specified for the given texts. It might be that the prompt is not directive enough or the texts are ambiguous for the model.

Then, the few-shot approach was used for the same purpose. This time, the prompt included not only an explanation of its role but also a few examples of classified sentences. Although the model does not have difficulty or hesitations in choosing an emotion, the evaluation metrics are not improved enough. The accuracy and the macro averages for the precision, recall and F1-score are almost the same as the ones of the zero-shot method.

The results showed that the model's performance on the test set (accuracy, precision, recall, and F1-score) was clearly lower than on the validation set. This could be due to many reasons. One possible reason is overfitting, especially when the fact that prompt adjustments were made based on validation performance is considered. Another possible reason is that the model has limited generalization ability. It is an LLM that was not trained for classification and it’s not a relatively large model. It means that it cannot perform as well on unseen data. In order to improve the performance of the model, more diverse training data can be used or the dataset size can be increased.

5.3 Embedding and Fine Tuning

5.3.1 BERT Embedding + Logistic Regression

It can be observed that the BERT embedding + Logistic Regression model performed moderately on the validation set. It achieved %60 accuracy and macro averages for precision, recall and F1-score are around 0.5. Although some classes like *joy* and *sadness* had relatively high F1-scores, others such as *love* and *surprise* showed weak performance. This may be due to the frozen embeddings which means that the BERT model was not updated during training and could not learn how to classify emotions. Another reason might be that logistic regression is a simple linear classifier and it could not be able to predict relatively complicated emotions such as *surprise*. Finally, the class imbalance in the dataset could cause the model to perform poorly on predicting the less frequent emotion classes such as *surprise* and *love*, there are less examples to learn from in the dataset.

After evaluating the validation set results, the model was applied to the test set. The model's performance decreased for the *joy* class which is understandable since it has fewer examples than other classes. In contrast, the model performed as well on the validation set in predicting the *surprise* class even though it is the class with the least examples.

The conclusion is that the *embedding + logistic regression* method can also perform well on the unseen data set. The evaluation metrics of the test set are almost the same as the ones of the validation set which confirms this. Possible ways to improve this method is to use a larger dataset or try a different classification model.

5.3.2 Fine Tuning the BERT Model

To get better results, the BERT Model was fine-tuned on the dataset and it was concluded with strong results. The accuracy of the model reached %93 and the macro averages for the precision, recall and F1-score is 0.89. Classes with the most examples such as *sadness* and *joy* have high precision, recall and F1-scores, not surprisingly but even the classes with fewer examples such as *love* and *surprise* achieved good performance with F1 scores of 0.83 and 0.76, respectively.

This results demonstrates that training the BERT model on a specific dataset gives much better results. It is most likely because the model can learn or understand the emotional patterns in the text more effectively when it is trained on the specific task. When it is compared to the previous approach, using fixed embeddings, fine-tuning allows the model to adapt to the task and improve its performance.

6. Conclusion

In this project, we worked on solving the task of emotion classification on a given dataset using different techniques. We applied both classical machine learning methods and large language model (LLM) approaches. First, we used TF-IDF to vectorize the data and built models using logistic regression and SVC. These models performed well, reaching around 0.86 F1-score on both the validation and test sets. Then, we experimented with the Qwen 2.5-1.5B large language model. Although we saw small improvements in some metrics, the overall results were not successful. We first used a zero-shot technique and then tried to improve it with a few-shot prompt. Even though we provided example sentences to guide the model, there was no major improvement. The zero-shot model reached an average F1-score of 0.48, while the few-shot model achieved only 0.47. We concluded that these models likely failed due to dataset limitations, the small size of the LLM, and its difficulty in distinguishing between emotional categories.

Later, we used pre-trained BERT embeddings with logistic regression, which gave relatively better results, achieving an average F1-score of around 0.60. However, we suspected that the fixed nature of the embeddings, the lack of training data, and the simplicity of the classifier limited the performance. To improve the results, we fine-tuned a BERT model on our dataset. We formatted and tokenized the data properly before training. This approach gave us the best results in the entire project. The fine-tuned model reached an average F1-score of 0.93, and even the most difficult classes like "surprise" and "anger" reached precision, recall, and F1-scores close to 0.7 and 0.9, respectively.

During this project, we learned many important things about how to work with both classical machine learning models and large language models for emotion classification. One of the most important things we noticed was how important the dataset is. No matter which model we use, whether it's a small model we train ourselves or a big pre-trained model the quality and balance of the dataset make a big difference. For example, some emotions like “surprise” (3.6%) and “love” (8.2%) had fewer examples in the dataset, and most of our models had trouble predicting these emotions. This showed us that if a class has only a few examples, the model cannot learn it well. So, having a balanced dataset is very important for good results.

We also saw that how we split and use the dataset matters a lot. While working with the Qwen model, we got better results on the validation set than on the test set. This made us think that the model might have learned too much from the validation set, which caused lower performance on new data. We also learned that designing good prompts is not always easy, and even small changes in the prompt can affect the results. When using large models, we had to clean the output, speed up the process with batch sizes, and try different settings like number of epochs or batch size to get better results. Finally, we saw that preparing the data in the right way like formatting and tokenizing is very important when working with models like BERT. In the end, our best results came from the model we fine tuned ourselves. This showed us that training a model for a specific task often works better than using a general model with simple prompts.

7. References

- https://huggingface.co/google-bert/bert-base-uncased
- https://pandas.pydata.org/docs/
- https://www.tensorflow.org/learn?hl=tr
- https://docs.pytorch.org
- https://scikit-learn.org/0.21/documentation.html

8. Requested Deliverable Links

- Bert-base-uncased HF
- Trained Models - Google Drive